

Dokumentasjon for vaskeapp(2025)

Beskrivelse:

Jeg gått for å lage en ganske enkel vaskeapp for kollektivet. Her kan de legge til hver gang de tar oppvasken eller ansvarsområdet sitt. Den er laget for 4 brukere, og alle 4 for tildelt unike ansvarsområder for huset (badet, kjøkkenet, stua, gangen). Dette er oppstart vinduet, etterpå kommer du til en main side her kan du legge til hvis du har tatt oppvask eller ansvarsområdet ditt med datoen du gjorde det.

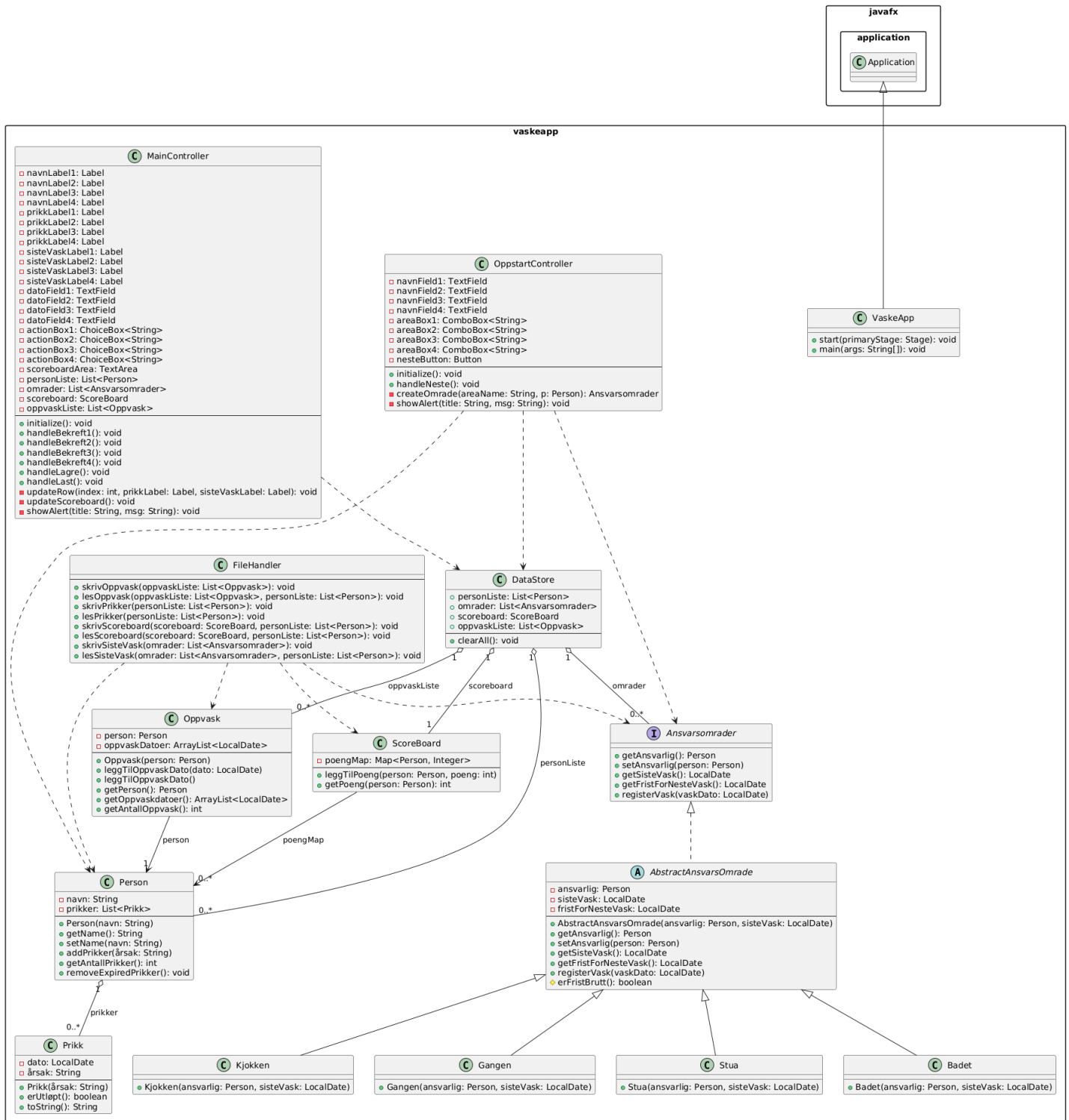
Jeg har laget en scoreboard system. Der hver gang du gjør oppvask eller vasker ansvarsområdet ditt tildeles det poeng. Hvis du vasker ansvarsområdet veies det 5 ganger så mye som hvis du tar oppvasken. Du har en frist på 2 uker på å ta ansvarsområdet ditt, hvis du unnlater det så vil det påføre prikk i systemet. Ved 5 prikker må du betale en bot (dette kan enkelt endres til å være lavere). Etter 3 mnd tid så vil prikken annulleres. Prikkene vises også på applikasjonen.

Du har muligheten til å lagre og laste statusen din ved applikasjonen. Dette blir da lagret i flere filer etter kategori for å ikke gjør det for avansert å hente ut data og legge det inn igjen (poeng.txt, oppvask.txt, sistevask.txt, prikker.txt). Dette er for å kunne avslutte programmet uten å miste alt.

Formålet med VaskeApp er å forenkle husholdningsrutiner og unngå diskusjoner om hvem som vasker mest eller minst, ved å gi en tydelig oversikt over alle oppgaver og poengscore.

Klassediagram:

VaskeApp - UML-klassediagram



1. Hvilke deler av pensum i emnet dekkes i prosjektet, og på hvilken måte? (For eksempel bruk av arv, interface, delegering osv.)
 - Interface brukes i AnsvarsOmrader. Den dekker hvilke grunnleggende metoder ansvarsområder må ha. Enhver klasse som implementer AnsvarsOmrader må for eksempel ha getAnsvarlig().
 - Den bruker arv, der klassene kjøkken, gangen, stua, badet arver fra abstrakte klassen AbstractAnsvarsOmrade. Dette er for å unngå å skrive repeterende kode.
 - Prosjektet viser innkapsling og validering: I klassen Person er navnefeltet privat, og vi sjekker i konstruktøren om navnet er gyldig (bruker isValidNavn()). Tilsvarende ligger poenglogikken i ScoreBoard, som beskytter tilstanden sin (et Map<Person, Integer>) ved å tilby metoder som leggTilPoeng.
 - Delegering sees ved at MainController tar seg av brukerinteraksjon (f.eks. når man trykker en knapp for å registrere vask), men videresender logikken til riktig klasse i modellen. For eksempel kalles ao.registerVask() når man registrerer at et rom er vasket.
 - Har laget også en måte håndtere filer på, der FileHandler bruker en try og catch metode for exceptions.
2. Dersom deler av pensum ikke er dekket i prosjektet deres, hvordan kunne dere brukt disse delene av pensum i appen?

Jeg kunne ha brukt Comparable/Comparator for å sortere personer etter poeng. For eksempel kunne man la Person implementere Comparable<Person> for å sortere en liste av Person-objekter etter antall prikker eller navn alfabetisk.
3. Hvordan forholder koden deres seg til Model-View-Controller-prinsippet? (Merk: det er ikke nødvendig at koden er helt perfekt i forhold til Model-View-Controller standarder. Det er mulig (og bra) å reflektere rundt svakheter i egen kode)

Den er ikke helt perfekt der. Har noe kode som kunne ha blitt implementert direkte i underliggende klassene (Person, AbstractAnsvarsOmrade osv.) som er implementert i kontrolleren. Jeg kunne gjort at feilmeldinger ikke lå i kontroll klassen, men ellers følger den MVP-prinsippene ganske greit.
4. Hvordan har dere gått frem når dere skulle teste appen deres, og hvorfor har dere valgt de testene dere har? Har dere testet alle deler av koden? Hvis ikke, hvordan har dere prioritert hvilke deler som testes og ikke? (Her er tanken at dere skal reflektere rundt egen bruk av tester)

For å teste koden er det viktig å teste kjerne prinsippene til koden eller hva du tenkte var viktigst for appen. Veien jeg gikk for dette var:

 - Person: Validering av navn, tillegg av prikker og sjekk av utløpte prikker. Om det var null/ugyldig navn.
 - ScoreBoard: Legge til poeng for en person, hente ut riktig poengsum, teste scenarioer med flere personer.
 - FileHandler: Skrive og lese fil for å se om data beholdes korrekt. For eksempel teste at Oppvask-datoer er like før og etter skriving/lesing.
 - AbstractAnsvarsOmrade: Teste at registerVask oppdaterer sisteVask og fristForNesteVask riktig, og eventuelt at fristen kan utløses slik at prikker gis.

Dette valgte jeg fordi disse kriteriene var kritiske for at appen jeg lagde skulle fungere og oppføre seg slik jeg forventet. Jeg vet ikke helt hvordan man får testet kontroller

eller fxml filen, så disse ble ikke testet. Var heller ikke så nødvendig, siden all data som blir gitt ut der kommer fra underliggende klassene og txt-filene som ble testet av JUnit testene. Hvis noen knappene ikke funket slik de skulle, så ser vi det ved manuell testing.